# Examination Management Portal — Architecture, Site Plan, Sitemap & DB Design

Oct 15, 2026 · @Fidele

The Examination Management Portal (EMP) is a role-based Flask + SQLite web app in which an Admin schedules exams, Examiners publish slots and grade, and Students book slots and see results. This document covers architecture, site plan, sitemap and database design for that build.

## 1. Scope and constraints

| Constraint from the brief | Design consequence |
|---|---|
| Flask back-end | Application factory + one blueprint per role |
| Jinja2, HTML, CSS, Bootstrap only | Server-rendered pages; Bootstrap is the only CSS framework |
| SQLite only, created in code | `db.create_all()` at startup + a seed step that inserts the Admin; no manual DB tools |
| JS must not run core requirements | Every action is a form POST or a link; JS only for optional extras (charts, client validation) |
| Admin pre-exists | Seeded on first run; no Admin registration route |
| Demo on a local machine | `pip install -r requirements.txt` then `python app.py`; no external services |

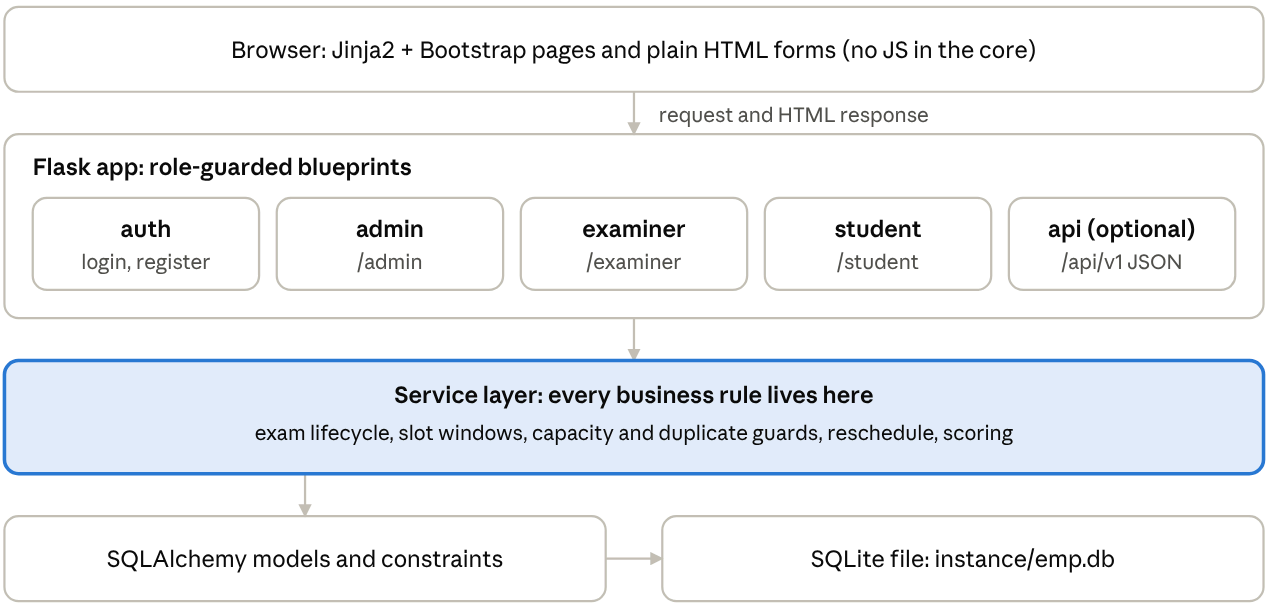
**Roles:** Admin (superuser), Examiner (needs Admin approval before the dashboard opens), Student (self-registers, active at once).

**Core flow:** Admin creates a Course, then an Exam with rubric criteria and four dates. In the slot-creation window the Examiner creates slots. In the booking window the Student books one. After the exam the Examiner scores each rubric criterion, and once results are published the Student sees marks and remarks.

## 2. System architecture

EMP is a single-process, server-rendered monolith in four layers: Jinja2 pages, role blueprints, a service layer that owns every business rule, and SQLAlchemy models over one SQLite file. Routes stay thin; rules live in services so they are enforced the same way from HTML forms and from the optional JSON API.

**Every request passes role blueprints and one service layer before SQLite**




request path · 4 layers, 5 blueprints

Read top to bottom: blueprints decide who may call what, the service layer decides whether the action is allowed, and models store the result.

| Layer | Responsibility | Tools |
|---|---|---|
| Presentation | Bootstrap pages, forms, flash messages, pagination, search boxes | Jinja2 templates, Bootstrap 5, HTML5 validation |
| Routing | URL → handler; reads the form, calls a service, renders or redirects | Flask blueprints: `auth`, `admin`, `examiner`, `student`, `api` |
| Services | Slot window checks, capacity, duplicate-booking guard, reschedule, examiner change, scoring | Plain Python modules, one transaction per action |

| Layer | Responsibility | Tools |
|---|---|---|
| Data | Models, constraints, indexes, seed | Flask-SQLAlchemy over SQLite (`instance/emp.db`) |

**Access control.** Flask-Login holds the session. A `role_required('admin' | 'examiner' | 'student')` decorator guards each blueprint. Examiner routes also pass an `approved_examiner_required` check, which sends pending users to a holding page. Object-level checks (an examiner may open only bookings in their own slots; a student only their own bookings) sit inside the services, not the templates.

**Security basics.** Passwords hashed with Werkzeug. CSRF tokens on every POST through Flask-WTF. SQLAlchemy parameterised queries only. Jinja auto-escaping stays on. Secret key and DB path come from config.

**No-JS core.** Booking, cancelling, slot creation, window open/close, evaluation and search are plain `<form method="post">` submissions with a redirect-after-POST. Server-side validation is authoritative; HTML5 attributes (`required`, `min`, `pattern`) are a convenience. JavaScript is limited to optional Chart.js statistics.

**Startup.** `create_app()` loads config, registers blueprints, runs `db.create_all()`, then seeds the Admin if none exists (credentials from environment or a documented default). This satisfies “database created programmatically”.

**Concurrency note.** SQLite serialises writers. A booking runs inside one transaction with a guarded update (`UPDATE slots SET available_seats = available_seats - 1 WHERE id = ? AND available_seats > 0`); zero rows changed means the slot is full, so overbooking cannot happen even with two simultaneous requests.

## 3. Site plan

The build splits into eight modules, ordered so each one can be demoed on its own. The project ships as one ZIP with a single root folder, as the brief requires.

## Modules

| # | Module | Delivers | Depends on |
|---|---|---|---|
| 1 | Foundation | App factory, config, models, `create_all`, Admin seed, base layout | none |
| 2 | Authentication | Login, student and examiner registration, logout, approval gate, role decorators | 1 |
| 3 | Courses and exams | Course CRUD, exam CRUD, status lifecycle, window dates | 2 |
| 4 | Rubrics and assignment | Criteria per exam, examiner approval, examiner-to-exam assignment | 3 |
| 5 | Slots | Examiner slot create/edit/delete inside the creation window, Admin slot view | 4 |
| 6 | Booking | Search and filter, book, cancel, duplicate and overbooking guards, history | 5 |
| 7 | Evaluation and results | Rubric scoring, remarks, publish, student results view | 6 |
| 8 | Admin control and extras | Reschedule, change examiner, global search, history, dashboards, charts, scorecard, JSON API | 6, 7 |

### Folder structure

```
emp/
├── app.py              # run: python app.py (creates the DB and the Admin)
├── models.py           # all tables
├── helpers.py          # role checks and the booking/slot rules
├── routes/
│   ├── auth.py
│   ├── admin.py
│   ├── examiner.py
│   └── student.py
├── templates/
│   ├── base.html
│   └── auth/  admin/  examiner/  student/
├── static/             # css, optional chart.js
├── requirements.txt
└── report/             # report, ER diagram
```

### Template plan

One `base.html` with a role-aware navbar. Shared macros cover form fields, status badges, pagination, the search bar and the empty-state card. Each role gets its own sidebar partial, so every page reuses the same shell.

### Build order and checkpoints

1. Foundation and auth: log in as seeded Admin, register an Examiner (blocked), approve them.
2. Course, exam, rubric: create a full exam with four dates and three criteria.
3. Slots and booking: Examiner makes slots, two Students book, third is refused at capacity.
4. Evaluation: score both Students, publish, check the Student results page.
5. Admin overrides and polish: reschedule, change examiner, search, charts, report and demo video.

## 4. Sitemap

The site has 4 areas and about 70 routes. Every page after login lives under its role prefix, and a user who opens another role's URL gets a 403 page. Actions marked POST are form submissions with no JavaScript.

## 4.1 Public and authentication

| Route | Method | Page / action |
|---|---|---|
| `/` | GET | Landing page with login and register links; signed-in users go to their dashboard |
| `/login` | GET, POST | One login form for all three roles; redirects by role |
| `/register/student` | GET, POST | Student sign-up (name, email, enrolment no., program, password) |
| `/register/examiner` | GET, POST | Examiner sign-up (name, email, phone, department); status starts Pending |
| `/examiner/pending` | GET | Holding page shown until the Admin approves (or after rejection / deactivation) |
| `/logout` | POST | End session |

## 4.2 Admin ( `/admin` )

| Area | Routes | Purpose |
|---|---|---|
| Dashboard | `/admin/dashboard` | Totals: courses, exams, examiners, students, slots, bookings; pending examiner count; exams by status |
| Courses | `/admin/courses` · `/new` · `/<id>/edit` · `/<id>/toggle` · `/<id>/delete` (POST) | List, create, edit, activate/deactivate, remove (blocked if exams exist) |
| Exams | `/admin/exams` · `/new` · `/<id>` · `/<id>/edit` · `/<id>/delete` (POST) | List with filters, create, detail, edit dates and fields |
| Exam lifecycle | `/admin/exams/<id>/open-slot-creation` · `/close-slot-creation` · `/open-booking` · `/close-booking` · `/complete` · `/publish-results` (all POST) | Move the exam through Draft → Slot Creation → Booking Open → Closed → Completed |
| Rubrics | `/admin/exams/<id>/rubrics` · `/new` · `/<rid>/edit` · `/<rid>/delete` (POST) | Criteria with max marks and weightage; running total check |

| Area | Routes | Purpose |
|---|---|---|
| Assignment | `/admin/exams/<id>/examiners` (GET, POST) | Assign or remove examiners for an exam |
| Examiners | `/admin/examiners` · `/<id>` · `/<id>/approve` · `/<id>/reject` · `/<id>/deactivate` · `/<id>/reactivate` | Approval queue, profile, workload |
| Students | `/admin/students` · `/<id>` | List, profile, full booking history |
| Slots | `/admin/slots` · `/<id>` · `/<id>/change-examiner` | All slots with filters; swap examiner in exceptional cases |
| Bookings | `/admin/bookings` · `/<id>` · `/<id>/reschedule` | All bookings; move to another slot of the same exam |
| Search | `/admin/search?q=&type=` | Students, examiners, exams, bookings by name or ID |
| History | `/admin/history` | Past exams, bookings and results, filter by course and date |
| Statistics (optional) | `/admin/reports` | Booking trend, examiner workload, completion rate |

## 4.3 Examiner ( `/examiner` )

| Area | Routes | Purpose |
|---|---|---|
| Dashboard | `/examiner/dashboard` | Assigned exams, slots created, students booked, pending evaluations |
| Exams | `/examiner/exams` · `/<id>` | Assigned exams with window dates and rubric |
| Slots | `/examiner/slots` · `/new?exam=<id>` · `/<id>/edit` · `/<id>/delete` (POST) | Create, edit, remove; allowed only in the creation window and before booking opens |

| Area | Routes | Purpose |
|---|---|---|
| Booked students | `/examiner/slots/<id>/students` | Roster for one slot |
| Evaluation | `/examiner/evaluations` · `/evaluate/<booking_id>` (GET, POST) | Pending and done list; score form with one field per criterion plus remarks |
| Profile | `/examiner/profile` | Edit contact details and password |

### 4.4 Student ( `/student` )

| Area | Routes | Purpose |
|---|---|---|
| Dashboard | `/student/dashboard` | Available exams and slots, upcoming exams, booked slots, published results |
| Exams | `/student/exams?q=&course=&type=&status=` · `/<id>` | Search, filter, detail with its open slots |
| Booking | `/student/slots/<slot_id>/book` (POST) · `/student/bookings` · `/<id>/cancel` (POST) | Book, list, cancel before the booking deadline |
| Schedule | `/student/schedule` | Calendar-style list of upcoming bookings |
| History | `/student/history` | Every booking ever made, with status changes |
| Results | `/student/results` · `/<booking_id>` · `/<booking_id>/scorecard` | Published marks, rubric breakdown, remarks, downloadable scorecard |
| Profile | `/student/profile` | Edit profile and password |

### 4.5 Optional JSON API ( `/api/v1` )

Read-only for exams, examiners, students and bookings, plus booking create/cancel for the logged-in student; details in section 7.

# 5. Database design

The schema has 12 tables in SQLite. One `users` table handles login for all roles; role details sit in 1:1 profile tables. Foreign keys are switched on with `PRAGMA foreign_keys=ON`, and every table has `id INTEGER PRIMARY KEY` and `created_at`.

**Bookings tie students to slots; evaluations hang off bookings**




ER diagram · 12 tables, 14 relationships

The diagram shows primary keys and the main foreign keys; full column lists follow in 5.1.

## 5.1 Tables

### users

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| email | TEXT | UNIQUE, NOT NULL, stored lower-case |
| password_hash | TEXT | NOT NULL |
| full_name | TEXT | NOT NULL |
| role | TEXT | CHECK in (admin, examiner, student) |
| is_active | BOOLEAN | default 1; 0 blocks login |
| created_at | DATETIME | |

### examiner_profiles

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | Examiner ID shown in the UI |
| user_id | INTEGER FK → users | UNIQUE, NOT NULL |
| phone | TEXT | |
| department | TEXT | NOT NULL |
| designation | TEXT | optional |
| status | TEXT | CHECK in (Pending, Approved, Rejected, Deactivated), default Pending |
| approved_by | INTEGER FK → users | NULL until approved |
| approved_at | DATETIME | |

### student_profiles

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| user_id | INTEGER FK → users | UNIQUE, NOT NULL |
| enrollment_no | TEXT | UNIQUE, NOT NULL; the Student ID shown in the UI |
| phone | TEXT | |
| program | TEXT | e.g. degree or batch |

### courses

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| code | TEXT | UNIQUE, NOT NULL |
| name | TEXT | NOT NULL |
| description | TEXT | |
| credits | INTEGER | optional extra |
| status | TEXT | CHECK in (Active, Inactive) |

### exams

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| course_id | INTEGER FK → courses | NOT NULL, ON DELETE RESTRICT |
| name | TEXT | NOT NULL |
| exam_type | TEXT | CHECK in (Viva, Practical, Project Demo, Assessment) |

| Column | Type | Notes |
|---|---|---|
| duration_minutes | INTEGER | CHECK > 0; also the default slot length |
| max_marks | INTEGER | CHECK > 0 |
| slot_create_start / slot_create_end | DATETIME | CHECK start < end |
| booking_start / booking_end | DATETIME | CHECK start < end; booking_start ≥ slot_create_end |
| status | TEXT | CHECK in (Draft, Slot Creation, Booking Open, Closed, Completed) |
| results_published | BOOLEAN | default 0; students see marks only when 1 |
| venue_mode | TEXT | optional (room or online link) |
| instructions | TEXT | optional |

### exam_examiners

Links an exam to the examiners allowed to create slots for it.

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| exam_id | INTEGER FK → exams | ON DELETE CASCADE |
| examiner_id | INTEGER FK → examiner_profiles | |
| assigned_by | INTEGER FK → users | Admin |
| UNIQUE | (exam_id, examiner_id) | |

### rubric_criteria

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |

| Column | Type | Notes |
|---|---|---|
| exam_id | INTEGER FK → exams | ON DELETE CASCADE |
| criterion_name | TEXT | NOT NULL; UNIQUE with exam_id |
| max_marks | NUMERIC | CHECK > 0 |
| weightage | NUMERIC | percent, CHECK between 0 and 100; all criteria of an exam sum to 100 |
| description | TEXT | |
| sort_order | INTEGER | display order |

### slots

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| exam_id | INTEGER FK → exams | NOT NULL |
| examiner_id | INTEGER FK → examiner_profiles | NOT NULL; current owner of the slot |
| slot_date | DATE | NOT NULL |
| start_time / end_time | TIME | CHECK start < end |
| capacity | INTEGER | CHECK ⩾ 1 |
| available_seats | INTEGER | CHECK between 0 and capacity |
| venue_mode | TEXT | optional |
| status | TEXT | CHECK in (Available, Full, Cancelled, Completed) |

### bookings

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| student_id | INTEGER FK → student_profiles | NOT NULL |
| slot_id | INTEGER FK → slots | NOT NULL |
| exam_id | INTEGER FK → exams | copied from the slot so the duplicate rule is one index |
| booking_date | DATETIME | NOT NULL |
| status | TEXT | CHECK in (Booked, Cancelled, Completed) |
| cancelled_at | DATETIME | set on cancel |
| UNIQUE (partial) | (student_id, exam_id) WHERE status IN ('Booked','Completed') | blocks duplicate bookings; a cancelled booking does not block a new one |

### booking_history

Append-only audit trail that gives each student a complete booking history.

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| booking_id | INTEGER FK → bookings | NOT NULL |
| action | TEXT | CHECK in (Booked, Cancelled, Rescheduled, Examiner Changed, Completed) |
| from_slot_id / to_slot_id | INTEGER FK → slots | for reschedules |
| acted_by | INTEGER FK → users | |
| note | TEXT | reason |

| Column | Type | Notes |
|---|---|---|
| created_at | DATETIME | |

### evaluations

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| booking_id | INTEGER FK → bookings | UNIQUE (one evaluation per booking) |
| examiner_id | INTEGER FK → examiner_profiles | who graded |
| total_marks | NUMERIC | computed from scores, scaled to exam max_marks |
| remarks | TEXT | overall feedback |
| status | TEXT | CHECK in (Draft, Submitted) |
| submitted_at | DATETIME | |

### evaluation_scores

| Column | Type | Notes |
|---|---|---|
| id | INTEGER PK | |
| evaluation_id | INTEGER FK → evaluations | ON DELETE CASCADE |
| criterion_id | INTEGER FK → rubric_criteria | |
| marks_awarded | NUMERIC | CHECK ≥ 0; app also checks ≤ criterion max |
| comment | TEXT | per-criterion feedback |
| UNIQUE | (evaluation_id, criterion_id) | |

Optional extra tables: `audit_log` (actor, action, entity, detail) for Admin changes, and `notifications` if email or in-app alerts are added.

## 5.2 Relationships

| Parent | Child | Cardinality | Meaning |
|---|---|---|---|
| users | examiner_profiles, student_profiles | 1 : 0..1 | A user has the profile that matches their role |
| courses | exams | 1 : N | A course has many exams |
| exams | rubric_criteria | 1 : N | Criteria belong to one exam |
| exams, examiner_profiles | exam_examiners | M : N | Examiners assigned to exams |
| exams | slots | 1 : N | Slots are created per exam |
| examiner_profiles | slots | 1 : N | One examiner owns each slot |
| slots | bookings | 1 : N | Up to `capacity` active bookings |
| student_profiles | bookings | 1 : N | A student has many bookings, one active per exam |
| bookings | booking_history | 1 : N | Every state change is logged |
| bookings | evaluations | 1 : 0..1 | Graded once |
| evaluations, rubric_criteria | evaluation_scores | M : N | One score per criterion |

## 5.3 Indexes

| Index | Serves |
|---|---|
| `slots(exam_id, slot_date, status)` | Student slot listing |
| `slots(examiner_id, slot_date)` | Examiner dashboard and clash check |
| `bookings(slot_id, status)` | Roster and seat count |

| Index | Serves |
|---|---|
| `bookings(student_id, status)` | Student history |
| `exams(status, booking_start)` | Search and filter |
| `users(email)`, `student_profiles(enrollment_no)` | Login and search |

### 5.4 Seed data

On first run: one Admin user ( `role='admin'` ). For the demo, an optional `seed_demo()` adds 2 courses, 2 exams with rubrics, 2 approved examiners and 5 students.

## 6. Business rules and status lifecycles

Every rule in the brief maps to one check in the service layer plus, where possible, a database constraint as a second line of defence.

| Brief requirement | Where it is enforced |
|---|---|
| Examiners create slots only in the creation period | `slot_service.create` : exam status is Slot Creation and now is inside `slot_create_start..end` ; examiner is assigned to the exam |
| Examiners update or remove slots before booking starts | Edit/delete allowed only while status is Slot Creation, now < `booking_start` , and the slot has no active bookings |
| Students book only in the booking period | `booking_service.book` : exam status is Booking Open and now is inside `booking_start..end` |
| No overbooking | Guarded `UPDATE ... WHERE available_seats > 0` plus `CHECK (available_seats >= 0)` |
| Slot closes at capacity | When `available_seats` reaches 0 the slot becomes Full; a cancel or reschedule reopens it |

| Brief requirement | Where it is enforced |
|---|---|
| No duplicate booking for one exam | Service pre-check plus the partial unique index on `bookings(student_id, exam_id)` |
| No clashing bookings (extra) | A student cannot hold two Booked slots whose date and time overlap |
| Cancel before the booking deadline | `cancel`: status is Booked and now < `booking_end`; seat is returned |
| Examiner evaluates only their own students | `evaluate` loads the booking and requires `booking.slot.examiner_id == current examiner`; otherwise 403 |
| Full booking history for every student | Every book, cancel, reschedule, examiner change and completion appends to `booking_history`; rows are never deleted |
| Admin reschedules a booking | Moves the booking to another slot of the same exam with a free seat, in one transaction, and logs it |
| Admin changes the assigned examiner | Updates `slots.examiner_id` (and `exam_examiners` if needed) to an approved examiner without a time clash; the old examiner loses access, the new one gains it; logged |
| Examiners need approval | Login succeeds, but every examiner route redirects to `/examiner/pending` unless status is Approved |
| Results visible only when published | Student result pages require `evaluations.status = Submitted` and `exams.results_published = 1` |

**Scoring.** For each criterion, `awarded / criterion max × weightage`; the sum, divided by 100 and multiplied by the exam's `max_marks`, gives `total_marks`. Rubric weightages must total 100 before an exam can leave Draft.

**Exam status.** Draft → Slot Creation (Admin opens creation) → Booking Open (Admin closes creation and opens booking) → Closed (Admin closes booking) → Completed (after evaluations; results can then be published). An Admin button moves the state; the date

windows must also hold, so a stale open state cannot bypass the deadline.

**Booking status.** Booked → Cancelled (student or Admin) or Booked → Completed (after the examiner submits the evaluation).

**Slot status.** Available ↔ Full as seats change; Cancelled if the examiner removes it after creation; Completed once the exam is Completed.

**Examiner status.** Pending → Approved or Rejected; Approved ↔ Deactivated. Deactivating an examiner is refused while they own future slots with bookings, until the Admin reassigns them.

## 7. Optional API and extras

The API is optional and reuses the same services as the HTML pages, so the rules above apply unchanged. It returns JSON and uses the logged-in session (or a token if you add one).

| Endpoint | Method | Access | Returns |
|---|---|---|---|
| `/api/v1/exams` | GET | any signed-in user | Exams, filterable by `course`, `type`, `status` |
| `/api/v1/exams/<id>` | GET | any signed-in user | Exam with rubric and open slots |
| `/api/v1/exams/<id>/slots` | GET | any signed-in user | Slots with `available_seats` |
| `/api/v1/examiners` | GET | Admin | Examiners with status |
| `/api/v1/students` | GET | Admin | Students |
| `/api/v1/bookings` | GET | Admin: all; Student: own; Examiner: own slots | Bookings |
| `/api/v1/bookings` | POST | Student | Book a slot ( `slot_id` ) |
| `/api/v1/bookings/<id>` | DELETE | Student (own), Admin | Cancel a booking |

**Extras worth adding after the core is done**, in order of value: Chart.js charts on the Admin reports page (booking trend, examiner workload, completion rate); downloadable scorecard as PDF; email on booking confirmation and result publication (Flask-Mail with a console fallback for local demos); CSV export of bookings.

**Report checklist.** The 5-page report needs the ER diagram from section 5, this endpoint table, the AI/LLM declaration, the frameworks list and the video link.